Experimenting with Word Vectorizer Models in Natural Language Processing

Subject: Machine Learning in Cyber Security

Authors: K.Yeshwanth(ch.sc.u4cys24023)

K.Abhiram (ch.sc.u4cys24027)

1. Word Vectorization in NLP — Overview & Significance

Word vectorization: Converts text into numerical vectors → essential for NLP.

- Captures **semantic meaning** → similar words have similar vectors.
- Types:
  - **BoW, TF-IDF** (frequency-based)
  - **GloVe, FastText** (embedding-based, semantic + syntactic)

Importance:

- Used in **classification, sentiment analysis, IR, translation, NER**
- Affects **accuracy, speed, generalization**
- **Pre-trained embeddings** enable **transfer learning**

---

2. Word Vectorization Methods

2.1 BoW (Bag of Words):

Represents text as word-count vectors; simple and fast but high-dimensional, ignores order/semantics, and treats words independently; used in spam detection and classification.

2.2. TF-IDF (Term Frequency – Inverse Document Frequency)

Improves BoW using TF × log(N/df), highlighting rare important words and reducing common ones; still sparse and order-agnostic; used in search and ranking.

2.3 GloVe (Global Vectors for Word Representation)

Dense embeddings (50–300d) using global co-occurrence capture semantics (e.g., king − man + woman ≈ queen) but cannot handle OOV words; used in sentiment analysis, NER, and transfer learning.

2.4 FastText (Facebook AI Research, 2017)

Extends Word2Vec using character n-grams, forming word vectors from subwords; handles OOV and misspellings, making it robust for noisy text; used in multilingual NLP and social media.

References:

Pennington et al. (2014), Bojanowski et al. (2017), Salton & Buckley (1988).

---

3. Dataset Description

- **Dataset Name:** Twitter Sentiment Dataset (Twitter_Data.csv)
- **Source:** Twitter — political tweets related to Indian politics (primarily centered around BJP/Modi era discourse)
- **Task Type:** Sentiment Analysis (3-class text classification)
- **Total Records:** 1,62,980 tweets
- **Columns:** 2 — clean_text (tweet content) and category (sentiment label)
- **Missing Values:** 4 null texts, 7 null labels — negligible, will be dropped during preprocessing
- **Text Length:** Average 124 characters per tweet; range 1–274 characters

Label Distribution:

Category	Sentiment	Count
1.0	Positive	72,250 (44.3%)
0.0	Neutral	55,213 (33.9%)
-1.0	Negative	35,510 (21.8%)

Sample Entries:

Negative (-1): “when modi promised minimum government maximum governance expected him begin the difficult job reforming...”; Neutral (0): “talk all the nonsense and continue all the drama will vote for modi...”; Positive (1): “what did just say vote for modi welcome bjp told you rahul the main campaigner...”.

Why this dataset is suitable:

- Large enough (162K samples) to produce statistically meaningful comparisons across all 4 vectorization methods
- Already partially cleaned (column named clean_text) — minimal preprocessing needed
- 3-class classification makes performance comparison across BoW, TF-IDF, GloVe, and FastText more informative than binary tasks
- Real-world noisy social media text — ideal for testing FastText's subword robustness and GloVe's semantic depth

---

4. Methodology

The experimental pipeline follows a structured approach from raw data ingestion to model evaluation.

4.1 Preprocessing

- **Dataset**: Twitter_Data.csv (162,980 tweets) with 3 classes — Negative (-1), Neutral (0), Positive (+1)
- **Null Handling**: Removed rows with missing values in *clean_text* or *category*
- **Label Fixing**: Converted category to integer for proper classification
- **Text Cleaning**:

Lowercasing (case-insensitive matching)

Removed URLs, mentions (@), and hashtags (#)

Removed non-alphabetic characters ([^a-z\s])

Normalized whitespace

- **Noise Reduction**:

  Removed stop-words (common non-informative words)

  Removed very short tokens (≤1 character)

- **Output**: Clean, meaningful text ready for vectorization

4.2 Model Training and Evaluation:

- **Classifier**: Logistic Regression — fast, interpretable, works with both sparse (BoW, TF-IDF) and dense (GloVe, FastText) features
- **Train/Test Split**: 80/20 with stratification to maintain class balance
- Reproducibility: random_state=42
- **Convergence**: max_iter=1000 to avoid solver warnings
- **FastText Setting**: C=5 to handle high-dimensional feature space
- **Timing**: Measured vectorization and training time separately using time.time()
- **Evaluation Metrics**:
  - Accuracy, Precision, Recall, F1-score (primary metric)
  - Used weighted average for class imbalance
- **Detailed Analysis**: classification_report() for per-class metrics
- **Results Storage**: Stored in a dictionary and displayed in a comparison table

---

5. Experimental Results

Actual Results (from Twitter_Data.csv)

Method	Accuracy	Precision	Recall	F1-Score	Vec Time	Train Time
BoW	**0.9485**	**0.9482**	**0.9485**	**0.9483**	2.26s	22.54s
TF-IDF	0.9235	0.9236	0.9235	0.9225	9.98s	17.34s
FastText	0.8791	0.8783	0.8791	0.8785	15.69s	47.53s
GloVe	0.5258	0.4969	0.5258	0.4735	4.41s	7.15s

---

6. Analysis

6.1 Performance Comparison

- BoW achieved highest performance (94.85% accuracy, F1: 0.9483) — best overall.
- TF-IDF ranked second (92.35%, F1: 0.9225) — weighting didn't significantly help on this dataset.
- FastText third (87.91%, F1: 0.8785) — works reasonably due to character n-grams.
- GloVe worst (52.58%, F1: 0.4735) — near-random due to random embeddings.
- Precision & Recall were balanced for BoW, TF-IDF, FastText.
- GloVe showed imbalance → biased toward majority classes.

Ranking: BoW > TF-IDF > FastText >> GloVe

6.2 Model Behavior:

BoW:

- Fast vectorization (**2.26s**), moderate training (**22.54s**).
- Uses raw word counts → key sentiment words dominate.
- Works best for Twitter due to frequent emotional keywords.

TF-IDF

- Slower vectorization (**9.98s**), faster training (**17.34s**).
- Reduces noise via weighting; captures phrases with bigrams.
- Slightly lower accuracy → bigrams add sparsity/noise in short text.

FastText

- Slowest vectorization (**15.69s**) and training (**47.53s**).
- Handles spelling variations (e.g., “luv”, “gr8”).
- Lower accuracy → subword units lose clear semantic meaning.

GloVe (Random)

- Fast vectorization (**4.41s**) and training (**7.15s**).
- Fails due to **random embeddings (no semantic meaning)**.
- Serves as a baseline → pretrained GloVe would perform much better (~85–92%).

6.3 Generalization:

- **BoW & TF-IDF** generalize best → Twitter sentiment is keyword-driven.
- Stratified split ensures fair comparison (no class bias).
- **FastText** handles unseen words but advantage is limited here.
- **GloVe** fails due to random initialization (no transferable patterns).
- **BoW shows no overfitting** → strong train-test consistency.

---

7. Time Complexity Analysis

The execution time involves two distinct phases: Vectorization and Model Training.

7.1 Vectorization Complexity

Bag of Words

- Time: **O(N × L)** — single pass over all documents.
- Fastest: **2.26s** for 130k+ samples.
- Uses optimized **CountVectorizer (C-backed sparse matrix)** → very low overhead.

TF-IDF

- Time: **O(N × L + V × N)** — TF + IDF computation.

- Bigram (1,2) increases tokens → slower (**9.98s**, ~4.4× BoW).
- sublinear_tf=True adds log scaling (minor overhead).

FastText (Character N-gram)

- Time: **O(N × L × W × G)**.
- Generates multiple n-grams per word → token explosion.
- Larger vocab (**50,000**) increases computation.
- Slowest: **15.69s**.

GloVe (Averaged Embeddings)

- Time: **O(N × L × D)**.
- Uses 100-dim vectors → mean computation adds cost.
- Moderate speed: **4.41s** (faster than TF-IDF/FastText, slower than BoW).

7.2 Model Training Complexity

BoW: (104k × 10k) sparse; 22.54s; slower due to large raw counts → more iterations.
TF-IDF: (104k × 10k) sparse; 17.34s; faster due to normalized values; reduces noise.
FastText: (104k × 50k) sparse; 47.53s (slowest); high dimensions → heavy computation; C=5 improves accuracy.
GloVe: (104k × 100) dense; 7.15s (fastest); fewer features → quick convergence.

Summary Table

Method	Features	Matrix	Time
BoW	10,000	Sparse	22.54s
TF-IDF	10,000	Sparse	17.34s
FastText	50,000	Sparse	47.53s
GloVe	100	Dense	7.15s

8. Trade-off Analysis

- **8.1 Performance vs Speed**
- **BoW: Best accuracy + fastest → most efficient**
- **TF-IDF: Slower + slightly worse → not beneficial here**
- **FastText: Slowest + lower accuracy → useful only for OOV cases**
- **GloVe: Fast but very low accuracy → ineffective**

- **8.2 Complexity vs Accuracy**
- **TF-IDF & FastText: More complex but no gain → diminishing returns**
- **GloVe: Most complex but worst → needs pretrained vectors**
- **Insight: Simple methods work best for short text**

- **8.3 Memory vs Richness**
- **BoW/TF-IDF: Moderate features → memory grows with vocab**
- **FastText: High memory (50k features) → no accuracy gain**
- **GloVe: Low memory (dense) → scalable**
- **Insight: Dense = memory efficient, Sparse = vocab dependent**

---

9. Deep Learning Context:

This study uses BoW, TF-IDF, FastText, and GloVe, but should also be compared with modern models like **BERT and transformers** for better contextual understanding.

Feature	BoW / TF-IDF	FastText / GloVe	BERT / Transformers

Context	No (global frequency)	Partial (local context windows)	Yes (full bidirectional context)
Semantics	No ("good" and "great" are distinct)	Yes (captures synonyms)	Yes (captures meaning, sarcasm, nuance)
Dimensionality	High and sparse (10,000+ dimensions)	Low and dense (100–300 dims)	Fixed and dense (768–1024 dims)
OOV Handling	None	Partial (FastText via n-grams)	None needed (subword tokenizer)
Training Speed	Near instant	Moderate	Extremely slow
Data Requirement	Low	Medium (pretrained)	High (pretrained on billions of tokens)
Interpretability	High	Low	Very Low

---

10. Conclusion

BoW achieved the best performance (94.85%) and fastest speed, proving simple methods work best for short text. TF-IDF added cost with no gain, FastText was slower with lower accuracy, and GloVe failed due to random embeddings.

Higher complexity did not improve results, and computation increased with feature size. Thus, **BoW/TF-IDF are best choices** for such tasks. Future work includes pretrained embeddings and transformer models.

---

11. GitHub Repository:

The full source code, data preprocessing scripts, and experimental logs can be found at:

https://github.com/yeshwanthkamuni/word_vectorization-for-NLP/tree/main

12. Sample Predictions:

Input Text	Prediction	Explanation
"This product is amazing and works perfectly"	Positive	Strong positive words ("amazing", "perfectly") → all models agree
"Worst purchase ever, completely useless"	Negative	Strong negative words ("worst", "useless") → clear prediction
"The product is okay, not great but not bad"	Positive	Negation not captured → slight positive bias
"I don't know what to think about this"	Neutral	No strong keywords → defaults to neutral

---

13. Division of Work

Member 1 — Review, Analysis & Report

- **Literature review on BoW, TF-IDF, FastText, GloVe**
- **Wrote theory, comparison, and conclusion**
- **Preprocessing + GloVe implementation**
- **Wrote analysis sections and compiled report**

Member 2 — Implementation & Presentation

- **Implemented BoW, TF-IDF, FastText pipelines**
- **Trained models, recorded timings & metrics**
- **Wrote performance, trade-off, and prediction sections**
- **Prepared slides, charts, demo, and documentation**

Submission for: 24CYS214 - Machine Learning in CYS